

Experiment no-1:

Experiment Name: Implementation of Static Members and this Keyword in Java Classes.

Task Name: BankAccount Class with Static Variable and Static Method

Objective

1. To understand the use of **static variables** in Java classes.
2. To create and initialize objects using **constructors**.
3. To implement a **static method** that can be called without creating an object.
4. To display account information using class methods.

Problem Analysis (Hybrid Model)

In banking systems, multiple accounts may belong to the same bank. Instead of storing the bank name separately for every account object, a **static variable** can be used because it is shared among all objects of the class. The program creates a BankAccount class with account information and a static bank name. A constructor initializes account data and a static method displays the bank name.

Input – Process – Output

Input:

Account number, account holder name, balance

Process:

Constructor initializes account information. Static variable stores bank name. Static method displays bank name and instance method displays account information.

Output:

Bank name and details of created bank accounts.

Algorithm

1. Create a class BankAccount.
2. Declare variables: accountNumber, accountHolderName, balance.
3. Declare static variable bankName = "Sonali Bank".
4. Create a constructor to initialize account details.
5. Create a static method showBankName() to display the bank name.

6. Create a method displayAccount() to show account information.
7. In the main class, create two objects of BankAccount.
8. Call the static method and display account information.

Source Code

```
class BankAccount {

    String accountNumber;
    String accountHolderName;
    double balance;

    static String bankName = "Sonali Bank";

    // Constructor
    BankAccount(String accountNumber, String accountHolderName, double balance)
    {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    // Static Method
    static void showBankName() {
        System.out.println("Bank Name: " + bankName);
    }

    // Method to display account information
    void displayAccount() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder Name: " + accountHolderName);
        System.out.println("Balance: " + balance);
        System.out.println("-----");
    }
}

public class BankAccountDemo {

    public static void main(String[] args) {
```

```
BankAccount.showBankName();

BankAccount acc1 = new BankAccount("1001", "Rahim", 50000);
BankAccount acc2 = new BankAccount("1002", "Karim", 75000);

acc1.displayAccount();
acc2.displayAccount();
}
}
```

Output (Example)

Bank Name: Sonali Bank

Account Number: 1001
Account Holder Name: Rahim
Balance: 50000.0

Account Number: 1002
Account Holder Name: Karim
Balance: 75000.0

Discussion

This program demonstrates how **static variables and static methods** work in Java. The bankName variable is shared among all objects of the class, meaning it is stored only once in memory. The static method showBankName() allows access to the bank name without creating an object. This concept is useful when certain information is common to all objects of a class.

Experiment no-2:

Task Name: LibraryBook Class with Constructor Using this Keyword

Objective

1. To understand the use of the **this keyword** in Java.
2. To create objects using a constructor.
3. To initialize instance variables using this.
4. To display book information using class methods.

Problem Analysis (Hybrid Model)

In object-oriented programming, constructors are used to initialize object data when an object is created. Sometimes the parameter names of the constructor are the same as the instance variables of the class. In such cases, the **this keyword** is used to refer to the current object's variables. This program creates a LibraryBook class where the constructor uses this to initialize the book details.

Input – Process – Output

Input:

Book title, author name, book ID

Process:

Constructor initializes the variables using this keyword and the method displays the information.

Output:

Book details displayed on the console.

Algorithm

1. Create a class LibraryBook.
2. Declare variables title, author, and bookID.
3. Create a constructor that initializes the variables using this.
4. Create a method displayBookInfo() to show book details.
5. In the main class, create two book objects.
6. Call the method to display book information.

Source Code

```

class LibraryBook {

    String title;
    String author;
    int bookID;

    // Constructor using this keyword
    LibraryBook(String title, String author, int bookID) {
        this.title = title;
        this.author = author;
        this.bookID = bookID;
    }

    // Method to display book information
    void displayBookInfo() {
        System.out.println("Book Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Book ID: " + bookID);
        System.out.println("-----");
    }
}

public class LibraryBookDemo {

    public static void main(String[] args) {

        LibraryBook book1 = new LibraryBook("Java Programming", "James Gosling",
101);
        LibraryBook book2 = new LibraryBook("Data Structures", "Mark Allen Weiss",
102);

        book1.displayBookInfo();
        book2.displayBookInfo();
    }
}

```

Output (Example)

Book Title: Java Programming
Author: James Gosling

Book ID: 101

Book Title: Data Structures

Author: Mark Allen Weiss

Book ID: 102

Discussion

The `this` keyword refers to the **current object of the class**. It helps differentiate instance variables from constructor parameters when both have the same name. Using `this` improves code clarity and ensures that object variables are correctly initialized.